- Try to ensure that data accessed by separate threads is sufficiently far apart to avoid false sharing.

Of course, that's not easy to apply to other data structures. For example, binary trees are inherently difficult to subdivide in any unit other than a subtree, which may or may not be useful, depending on how balanced the tree is and how many sections you need to divide it into. Also, the nature of the trees means that the nodes are likely dynamically allocated and thus end up in different places on the heap.

Now, having data end up in different places on the heap isn't a particular problem in itself, but it does mean that the processor has to keep more things in cache. This can actually be beneficial. If multiple threads need to traverse the tree, then they all need to access the tree nodes, but if the tree nodes only contain *pointers* to the real data held at the node, then the processor only has to load the data from memory if it's actually needed. If the data is being modified by the threads that need it, this can avoid the performance hit of false sharing between the node data itself and the data that provides the tree structure.

There's a similar issue with data protected by a mutex. Suppose you have a simple class that contains a few data items and a mutex used to protect accesses from multiple threads. If the mutex and the data items are close together in memory, this is ideal for a thread that acquires the mutex; the data it needs may well already be in the processor cache, because it was just loaded in order to modify the mutex. But there's also a downside: if other threads try to lock the mutex while it's held by the first thread, they'll need access to that memory. Mutex locks are typically implemented as a read-modify-write atomic operation on a memory location within the mutex to try to acquire the mutex, followed by a call to the operating system kernel if the mutex is already locked. This read-modify-write operation may well cause the data held in the cache by the thread that owns the mutex to be invalidated. As far as the mutex goes, this isn't a problem; that thread isn't going to touch the mutex until it unlocks it. However, if the mutex shares a cache line with the data being used by the thread, the thread that owns the mutex can take a performance hit *because another thread tried to lock the mutex*!